

Reviewer Pack — Coboundary-Vanishing Threshold: HX¹ Triviality Below the Di...

Entry 264c40c0d040 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 03:39:41 UTC

Coboundary-Vanishing Threshold: HX¹ Triviality Below the Diameter-to-Depth Ratio for Parity-on-a-Tree

Entry ID: 264c40c0d040 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 03:39:41 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW) framework / coarse (Roe) cohomology and geometric group theory applied to circuit complexity **Field B** (complexity object): Formula depth lower bounds for the recursive-parity (balanced AND/OR-tree of XORs) function in the NC¹ vs. AC⁰[$\oplus$] regime, expressed via the coarse 1-cohomology $HX^1(X_f)$ and the Roe-trace pairing on $C_u^*[X_f]$

Statement:

Let $f_n = \oplus\text{-tree}_n$ be the balanced binary tree of XOR gates on $n = 2^k$ inputs, with KW-metric space $(X_{\{f_n\}}, d_{\{f_n\}})$. Define the coboundary $\delta : C^0_{\text{coarse}}(X_{\{f_n\}}) \rightarrow C^1_{\text{coarse}}(X_{\{f_n\}})$ and the propagation-R Roe subalgebra $C_u^*[X_{\{f_n\}}]_R$. Then there exists a universal constant $\alpha \in (0,1)$ such that, for all $R \leq \alpha \cdot \log_2 n$, every coarse 1-cocycle c with propagation $\leq R$ is a coboundary (so its class in HX^1 is trivial), while for $R \geq k = \log_2 n$ there exist nontrivial classes c whose Roe-trace pairing satisfies $\log| \langle c, T \rangle | / \log(\text{propagation}(T)) \geq k - O(1)$. Consequently $\kappa(f_n) \geq \log_2 n - O(1)$, matching $\text{depth}(f_n) = \Theta(\log n)$ up to constants and certifying axiom A1 sharply for an explicit recursive function.

Rationale (proposer's reasoning):

This conjecture directly tests axiom A1 (Coarse-KW link) together with A5 (Roe-index rigidity) on the canonical recursive function whose formula depth is exactly known ($\text{depth}(\oplus\text{-tree}_n) = \log_2 n$). The KW protocol for the parity tree must descend the tree to find a differing leaf, so the controlled-cover multiplicity must grow by 1 per level; this should manifest as a vanishing-threshold for HX^1 at propagation $R \approx \log_2 n$, with cocycles at smaller propagation forced to be coboundaries. Verifying both the vanishing below the threshold and the nonvanishing above gives a sharp quantitative version of A1 on a function whose depth we can independently certify, providing a sanity check that κ is neither overshooting nor undershooting the true formula depth in a controlled recursive setting.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_85a254b4a0, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 71a9e7de2d29d92a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{4, 8, 16, 32\}$ (one deterministic seed for $X_{\{f_n\}}$ construction, 5 random seeds for Roe operator T per n), pre-register: (a) vanishing threshold $R(n) := \max\{R : \dim HX^1_R = 0\}$ satisfies $|R(n) - \lfloor 0.5 \cdot \log_2 n \rfloor| \leq 1$ for ≥ 3 of 4 n ; (b) $\dim HX^1_R \geq 1$ for all $R \geq \log_2 n$; (c) Pearson correlation of empirical slope $\log|(c, T)|/\log R$ vs $\log_2 n$ across the 4 n values ≥ 0.95 , with mean slope $\geq \log_2 n - 2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.88	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - coarse cohomology Roe algebra Karchmer-Wigderson formula depth lower bounds - uniform Roe algebra propagation cocycle circuit complexity NC1 parity tree - coarse geometry HX^1 coboundary recursive parity formula depth AC0 XOR

Top relevant hits considered: - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/2109.13292v1>] The Hochschild Cohomology of Uniform Roe Algebras - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1906.11725v2>] On the uniform Roe algebra as a Banach algebra and embeddings of ℓ_p uniform Roe algebras - [<http://arxiv.org/abs/0906.0239v3>] Cocycle deformations for Hopf algebras with a coalgebra projection - [<http://arxiv.org/abs/1907.01259v4>] Coboundary and cosystolic expansion from strong symmetry - [<http://arxiv.org/abs/2602.24220v1>] Comparing Classical and Quantum Variational Classifiers on the XOR Problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys
import json
from collections import defaultdict

def xor_tree(n):
```

```

if n == 1:
    return [0]
left = xor_tree(n // 2)
right = xor_tree(n // 2)
return [left[i] ^ right[i] for i in range(n)]

def hamming_distance(x, y):
    return sum(xi != yi for xi, yi in zip(x, y))

def bfs_diameter(tree, root):
    queue = [(root, 0)]
    visited = set([root])
    max_dist = 0
    while queue:
        node, dist = queue.pop(0)
        max_dist = max(max_dist, dist)
        for neighbor in range(len(tree)):
            if tree[node] ^ tree[neighbor] == 1 and neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))
    return max_dist

def build_cochain_space(tree, R):
    n = len(tree)
    C0 = [[0 for _ in range(n)] for _ in range(n)]
    C1 = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if hamming_distance(i, j) <= R:
                C0[i][j] = C0[j][i] = tree[i] ^ tree[j]
                C1[i][j] = C1[j][i] = random.choice([-1, 1])
    return C0, C1

def smith_normal_form(matrix):
    n = len(matrix)
    A = [row[:] for row in matrix]
    R = range(n)
    C = range(n)
    for k in range(n):
        pivot_row = max(R[k:], key=lambda r: abs(A[r][k]))
        A[pivot_row], A[R[k]] = A[R[k]], A[pivot_row]
        pivot_col = max(C[k:], key=lambda c: abs(A[R[k]][c]))
        A[R[k]][pivot_col], A[R[k]][C[k]] = A[R[k]][C[k]], A[R[k]][pivot_col]
        for r in R:
            if r != k:
                factor = A[r][k] / A[k][k]
                for c in C:
                    A[r][c] -= factor * A[k][c]
    return [sum(abs(A[i][j]) for j in C) for i in R]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 8, 16, 32]
    results = []
    for n in n_values:
        tree = xor_tree(n)

```

```

diameter = bfs_diameter(tree, 0)
R_max = math.floor(0.5 * math.log2(n))
HX1_R = [0] * (R_max + 1)
for R in range(R_max + 1):
    C0, C1 = build_cochain_space(tree, R)
    delta_0 = [[sum(C0[i][j] * C0[j][k] for j in range(n)) for k in range(n)] for i in range(n)]
    delta_1 = [[sum(C1[i][j] * C0[j][k] for j in range(n)) for k in range(n)] for i in range(n)]
    ker_delta_1 = [row[:] for row in delta_1]
    im_delta_0 = [row[:] for row in delta_0]
    for r in range(R_max + 1):
        if r != R:
            factor = sum(ker_delta_1[r][c] * im_delta_0[c][k] for c in range(n)) / sum(im_delta_0[r][k] for k in range(n))
            ker_delta_1[r][k] -= factor * im_delta_0[r][k]
    HX1_R[R] = len(smith_normal_form(ker_delta_1)) - len(smith_normal_form(im_delta_0))

if all(HX1_R[:R_max + 1]) and any(HX1_R[R_max + 1:]):
    slope, intercept = 0, 0
    count = 0
    for R in range(R_max + 1, len(HX1_R)):
        if HX1_R[R]:
            slope += math.log(abs(HX1_R[R])) / math.log(R)
            intercept += math.log(R)
            count += 1
    if count > 0:
        slope /= count
        intercept /= count
        results.append({
            "n": n,
            "slope": slope,
            "intercept": intercept
        })

if not results:
    return {
        "metric_name": "vanishing_threshold",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_slope = sum(result["slope"] for result in results) / len(results)
std_slope = math.sqrt(sum((result["slope"] - mean_slope) ** 2 for result in results) / len(results))
correlation = sum((result["slope"] - mean_slope) * (math.log2(result["n"]) - mean_log_n) for result in results) / len(results)
mean_log_n = sum(math.log2(result["n"]) for result in results) / len(results)

return {
    "metric_name": "vanishing_threshold",
    "metric_value": mean_slope,
    "instances_tested": len(results),
    "conjecture_holds": correlation >= 0.95 and mean_slope >= math.log2(n_values[0]) - 2,
    "counterexample": "" if correlation >= 0.95 else f"correlation={correlation}, mean_slope={mean_slope}"
}

if __name__ == "__main__":

```

```

seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]
results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {json.dumps(result)}")
    results.append(result)

mean_slope = sum(result["metric_value"] for result in results) / len(results)
std_slope = math.sqrt(sum((result["metric_value"] - mean_slope) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation too low\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e3dd4e.py", line 133, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e3dd4e.py", line 73, in run_trial
    tree = xor_tree(n)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e3dd4e.py", line 20, in xor_tree
    left = xor_tree(n // 2)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7e3dd4e.py", line 22, in xor_tree
    return [left[i] ^ right[i] for i in range(n)]
            ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in the xor_tree construction before any data on $R^*(n)$, $\dim HX^1_R$, or slope correlations was produced, so neither the support nor falsification

conditions can be evaluated. | next: Fix the xor_tree recursion (the base case and/or input length handling causing the out-of-range index) and rerun the pre-registered protocol across $n \in \{4,8,16,32\}$.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	8930
2	novelty	claude_max	opus	0	0	3770
3	novelty	claude_max	opus	0	0	12149
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18869
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18445
6	judge	claude_max	opus	0	0	3986

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 66149 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in *research/audit/264c40c0d040.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/264c40c0d040.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/264c40c0d040.tar.gz* (if generated)